Lab Assignment Memo: Containerize the Production Flow

Assignment Title

Containerize the Production Flow

Timebox

Keep the scope simple. The goal is to prove that your existing VM-based production flow can run correctly in Docker Compose while preserving the most important reliability and networking properties.

Objective

Migrate your existing three-service system into a **Docker Compose environment**.

Your containerized system should run:

- Nginx
- Service A
- Service B
- Service C

The flow should remain:

```
Client
  -> Nginx
  -> Service A
  -> Service B
  -> Service C
  -> Service A callback
  -> Client response
```

What Must Be Preserved

Your Docker Compose version must preserve these core properties:

1. **Nginx is the only public entry point**
2. **Services B and C are internal-only**
3. **Services communicate using Docker Compose service names**
4. **The full A → B → C → A callback flow works**
5. **Logs can be viewed with** `docker compose logs`
6. **One request ID can be traced through the services**
7. **Stopping Service B produces a clear failure and recovery path**

Key Concept Shift

VM version	Docker Compose version
systemd starts services	Compose starts containers
`/etc/hosts` service names	Compose DNS service names
`journalctl` logs	`docker compose logs`
UFW + bind addresses	Docker networks + published ports
VM restart policy	Compose restart policy

The production ideas are the same. The runtime changes.

Required Deliverables

Each group should add the following to the repo:

Deliverable	Required?	Notes
`docker-compose.yml`	Yes	Defines Nginx, Service A, Service B, and Service C.
Dockerfile(s)	Yes	One per service, or one shared Dockerfile if cleanly explained.
Nginx config	Yes	Nginx should route public traffic to Service A only.
`.dockerignore`	Yes	Exclude `venv`, `.git`, caches, logs, and local files.
README update	Yes	Add a short “Run with Docker Compose” section.
Validation evidence	Yes	Add `docs/CONTAINER_VALIDATION.md` or screenshots/ command output.

Required Compose Rules

1. Only Nginx should publish a host port

Good:

```
nginx:
  ports:
    - "8080:80"
```

Avoid publishing Service B or Service C:

```
service-b:
  ports:
    - "3002:3002"
```

Service B and Service C should be reachable only inside the Docker Compose network.

2. Use Compose service names

Inside containers, do not call other services using `localhost`.

Use:

```
http://service-b:3002
http://service-c:3003
http://service-a:3001/callback
```

Do not use:

```
http://localhost:3002
http://127.0.0.1:3002
http://<host-ip>:3002
```

Inside a container, `localhost` means that same container.

3. Logs should go to stdout/stderr

You should be able to see logs using:

```
docker compose logs
docker compose logs service-a
docker compose logs service-b
docker compose logs service-c
docker compose logs nginx
```

Do not hide important logs only inside files in the container.

4. Add simple restart policies

Each application service should include a restart policy, for example:

```
restart: unless-stopped
```

or:

```
restart: on-failure
```

You only need to explain briefly why you chose it.

Minimum Validation Tests

Your `docs/CONTAINER_VALIDATION.md` should show command output or screenshots for the tests below.

1. Start the system

```
docker compose up --build -d
```

2. Confirm containers are running

```
docker compose ps
```

Expected:

```
nginx
service-a
service-b
service-c
```

All should be running.

3. Test public entry point

Use your group's public route.

Example:

```
curl -i http://localhost:8080/health
```

or:

```
curl -i http://localhost:8080/<your-public-route>
```

Expected: HTTP 200 or the expected successful application response.

4. Prove B and C are not directly exposed

From the host:

```
curl -i --connect-timeout 3 http://localhost:3002/health
curl -i --connect-timeout 3 http://localhost:3003/health
```

Expected: connection refused, timeout, or no route.

5. Prove internal service discovery works

From inside the Docker Compose network:

```
docker compose exec service-a curl -i http://service-b:3002/health
docker compose exec service-b curl -i http://service-c:3003/health
```

Expected: HTTP 200.

6. Trace one request

Send one request with a known request ID:

```
curl -i http://localhost:8080/<your-public-route>
  -H "X-Request-ID: demo-container-001"
```

Then check logs:

```
docker compose logs | grep demo-container-001
```

Expected: the same request ID appears in Service A, Service B, Service C, and preferably Nginx.

7. Stop Service B and observe failure

```
docker compose stop service-b
```

Send the request again:

```
curl -i http://localhost:8080/<your-public-route>
  -H "X-Request-ID: fail-service-b-001"
```

Expected:

- The request fails cleanly
- The response is understandable
- Service A logs the failure
- The system recovers after Service B is started again

Recover:

```
docker compose start service-b
```

Then run the successful request again.

README Update

Add a short section called:

```
Running with Docker Compose
```

Include:

1. How to start the system
2. How to test the public route
3. How to prove B and C are internal
4. How to view logs
5. How to stop and restart a service
6. How to shut everything down

Submission Requirements

Submit:

1. GitHub repository link
2. Commit SHA used for review
3. `docker-compose.yml`
4. Dockerfile(s)
5. Nginx config
6. Updated README
7. `docs/CONTAINER_VALIDATION.md`

What Not To Overdo

Do not spend the whole lab on perfect production hardening.

For this lab, you do **not** need to implement:

- Kubernetes
- CI/CD
- image scanning
- multi-stage builds, unless you already know how
- complex secrets management
- full observability stack
- Prometheus/Grafana
- service mesh
- TLS

Focus on the basics:

> Can the same production flow run correctly in containers, with Nginx as the only public entry point, internal service discovery, working logs, tracing, and a clear failure test?

Final Instruction

The key question is:

> Did you preserve the production behavior when the runtime changed from systemd on a VM to containers under Docker Compose?